

Breve guia de sobrevivência com Terraform

Breve guia de sobrevivência com Terraform

[View on GitHub](#)

Esse material foi inicialmente postado em <https://dev.to/tuxpilgrim/breve-guia-de-sobrevivencia-com-terraform-ggn>.

Só pra recapitular, o que é o Terraform mesmo?

Baseado na definição da Hashicorp (em uma tradução literal):

O Terraform é uma ferramenta de infraestrutura como código criado pela Hashicorp que permite definir recursos locais e/ou em nuvem em arquivos de configuração legíveis por humanos que você pode versionar, reutilizar e compartilhar. Você pode então usar um fluxo de trabalho consistente para provisionar e gerenciar todo ciclo de vida da sua infraestrutura.

A linguagem usada para escrever esses arquivos é o HCL (*Hashicorp Configuration Language*), mas também pode ser usado JSON.

Mais detalhes e a definição completa, você pode ver [aqui](#).

Por onde começar?

É possível começar com os tutoriais oficiais na [plataforma da Hashicorp](#), ao menos para casos de uso mais comuns (com provedores de nuvem) tem bastante coisa:

- [Terraform com AWS](#)
- [Terraform com Azure](#)
- [Terraform com Google Cloud](#)

Também existem alguns cursos/tutoriais gratuitos no Youtube em português(Brasil), como:

- [Terraform 1.0 - do básico a módulos via Caio Delgado](#)
- [Terraform 101 - Instalação e Deploy na AWS via Caio Delgado](#)
- [Curso completo - do básico ao menos básico via Igor Sousa](#)

Outras recomendações no Youtube, em inglês:

- [Learn Terraform \(and AWS\) by Building a Dev Environment vi FreeCodeCamp](#)

- [Learn Terraform with Azure by Building a Dev Environment via FreeCodeCamp](#)

E por último, e não menos importante, para quem gosta de livros, deixo a recomendação do [Terraform Up & Running](#).

Agora que já sei usar (ao menos o básico), onde posso aprender as melhores práticas?

A forma de uso do Terraform muitas vezes varia de organização pra organização, mas existem alguns guias que valem a pena serem seguidos (ou ao menos adaptados para a criação de uma melhor prática interna):

- [Práticas recomendadas para usar o Terraform via Google Cloud](#)
- [Livro aberto com recomendações de melhores práticas](#)
- [Guia completo de uso do Terraform via GruntWork¹](#)

Também existem algumas ferramentas/utilitários que ajudam a fazer o *enforcement* (garantia) dessas melhores práticas, como por exemplo:

- [Conftest](#): utilitário para escrever testes estruturados (políticas) usando o Open Policy Agent.
- [Pre-commit Terraform](#): git hook com alguns plugins para Terraform, ajudam a garantir que está tudo dentro do esperado antes do *push* para o repositório.

¹Valeu pelo lembrete desse link [Pablo](#).

Aqui também é importante citar que existem dois grandes grupos em uma discussão infinita quando se fala em organização dos projetos no Terraform:

- [time organiza por workspaces \(me refiro a workspace no Terraform OSS, que é diferente do Terraform Cloud\)](#)
- [time organiza por diretórios, com Terragrunt](#)

Recomendo ler sobre as duas opções e escolher a que traz mais benefícios para o seu cenário.

Se Terraform é infra como código, quais opções eu tenho para testar meu código?

O tópico testes de infraestrutura como código ainda tem muito a ser discutido/melhorado, mas até agora, as opções que temos disponíveis são:

- [Terratest](#): Framework de testes para Terraform, os testes devem ser escritos em Golang.

- **Kitchen Terraform**: Plugins do Test Kitchen para testar código Terraform, testes devem ser escritos em Ruby.
- **Terraform Tests**: Recurso nativo do Terraform que permite que os testes sejam escritos em HCL e executados com o comando `terraform test`.

Mais especificamente para AWS existe o **localstack**, que é basicamente um framework para simular algumas funcionalidades da AWS localmente.

Deixo aqui também um artigo bem interessante da Hashicorp sobre o assunto:

<https://www.hashicorp.com/blog/testing-hashicorp-terraform>.

Existem ferramentas de segurança para fazer análise estática (ou algum outro tipo) do meu código Terraform?

Existem algumas ferramentas opensource que fazem análise estática e de erros de configuração no código e lint. As mais usadas:

- **tfsec**: ferramenta para análise de código estático e erros de configuração em código Terraform.
- **checkov**: ferramenta para análise de código estático de IaC (infra como código), serve também para CloudFormation, Kubernetes e outros.
- **tflint**: Linter para Terraform, serve para avisar sobre problemas com sintaxe, erros nos principais provedores de nuvem, garantir boas práticas e outras coisas.

Algum material de referência relacionado a Terraform no pipeline de CI/CD?

Falando das plataformas de CI/CD mais usadas, todas tem workflows reutilizáveis para Terraform.

- **Github Actions**
- **Gitlab**
- **CircleCI**

Uma outra abordagem é usar o **Atlantis** para automatizar a execução dos comandos via Pull Requests (Merge Requests).

E caso eu já tenha recursos criados (manualmente), consigo importar para código HCL?

Esse normalmente é um processo doloroso (dependendo da quantidade de recursos), mas existem alguns utilitários que podem ajudar nisso:

- [Terraformer](#): tem suporte para os maiores provedores de nuvem (GCP, AWS e Azure) e outros serviços (como PagerDuty).
- [Aztfy](#): importa especificamente recursos da Azure.

Comandos interessantes da CLI do Terraform

- [terraform console](#): abre uma CLI interativa pra validar expressões.
- [terraform graph](#): gera uma visualização da configuração ou de um `plan`.

Outras ferramentas que podem ser úteis

- [Infracost](#): Estimativas de custos nos Pull Requests.
- [Tfswitch](#): ferramenta de CLI pra gerenciar versões do Terraform.
- [Terraboard](#): dashboard web para inspecionar states do Terraform.
- [Terraform Visual](#): ferramenta para ter visualização interativa das mudanças em um `plan`.
- [Terraform-docs](#)²: utilitário para gerar documentação para os módulos Terraform.
- [K2tf](#)³: utilitário para converter arquivos yaml do kubernetes para HCL.
- [IAM JSON para Terraform HCL](#)³: utilitário para converter políticas IAM (AWS) em .json para HCL.

²: Valeu pelo lembrete [Arielson](#)

³: Valeu pelo lembrete [Isac](#)

Contribuições são mais que bem vindas, só abrir um Pull Request se achar algo que deveria estar aqui e não está :)

[guia-rapido-terraform-ptbr](#) is maintained by [edsoncelio](#).

This page was generated by [GitHub Pages](#).